

Distributed Cache – Redis

Contributors

- Prakarsh Jain (2024201055)
- Vanshika Singh (2024202008)
- Ramachandran Moorthy (2024201063)

Problem Statement

Modern applications require high-speed access to frequently used data. Traditional databases often become bottlenecks due to high read/write loads, leading to performance degradation and increased latency. To address this, in-memory caching systems like Redis have become essential for improving application responsiveness. However, existing solutions may not always fit specific use cases due to cost, customization needs, or operational constraints.

This project aims to build a distributed, fault-tolerant, in-memory key-value store inspired by Redis that:

- Stores and retrieves data efficiently to reduce database load
- Ensures availability through sharding and replication
- Handles cache eviction
- Supports persistence mechanisms
- Implements compression to optimize memory usage
- Supports service discovery and load balancing for distributed setups
- Ensures fault tolerance and automatic failover when nodes go down
- Includes fallback to a SQL database when data is not found in the cache
- Logs and monitors system performance metrics

Technological Framework

- **Python** for robust and rapid development
- **gRPC** for high-performance inter service communication
- **Consul** for service discovery and load balancing

- **Protobufs** for efficient serialization
- **Prometheus, Grafana** for efficient logging and monitoring
- **MySQL** for backend database
- **Snappy / LZ4 / zlib** for compression
- **Docker** for containerization

Key Deliverables

1. Core Caching System
 - a. Key-Value Storage Engine
 - b. Persistence Mechanism
 - c. Data Compression
2. Distributed Architecture & Data Management
 - a. Sharding (Partitioning)
 - b. Replication (Master-Slave Replication)
 - c. Failover handling
3. Communication
 - a. gRPC-based Communication
4. High Availability
 - a. Consistency & Availability
 - b. Eviction Policies
 - c. Request Routing & Load Balancing
 - d. Efficient Logging and tracking using Prometheus, Grafana
5. Persistence & Recovery
 - a. Crash Recovery & Backup
 - b. Database Snapshot Management

Implementation Roadmap

Week 1: Core System & Infrastructure Setup

- Set Up Development Environment
- Implement Core Caching System
- Persistence Mechanism
- gRPC Service & Communication

Week 2: Distributed Architecture & Replication

- Sharding & Partitioning
- Replication & Failover Handling
- Service Discovery & Load Balancing
- Compression Mechanism

Week 3: Optimization & Fault Tolerance

- Eviction Policies
- High Availability & Consistency
- Efficient Logging and monitoring using Prometheus, Grafana

Week 4: Testing, Recovery, & Final Deployment

- Scale testing (evaluating metrics such as latency, throughput, cache hit ratio under high loads)
- Comparison of performance over normal functioning (without cache)
- Crash Recovery & Backup
- Database Fallback Mechanism
- Containerize services using Docker